



**Tecnicatura Universitaria en Procesamiento y Explotación
de Datos**

Licenciatura en Bioinformática

ALGORITMOS Y ESTRUCTURAS DE DATOS

Trabajo Práctico N°2

Fecha de entrega: 1/11/2024

**Autores: Renzo Ortega Lujan, Catriel Enrique,
Fernando Benedetich**

Los objetivos de nuestro TP son:

- Aplicar conceptos teóricos sobre estructuras jerárquicas, grafos y sus algoritmos asociados.

Problema 1:

Aquí se nos pide alterar una sala de emergencia ya diseñada, la cual atiende según el orden de llegada y sin importar el nivel del riesgo de los pacientes (siendo 1 crítico, 2 moderado y 3 bajo). De modo que atienda primero a los pacientes con nivel de riesgo 1, luego los paciente con 2, y por último los con 3; y en caso de haber varios pacientes con un mismo nivel de riesgo (ya sea varios críticos o varios moderados y así), se rige por un segundo criterio de elección elegido por nosotros, que elegimos el orden de llegada, es decir, nuestra sala de emergencias atenderá primero a los más críticos y en caso de haber varios con un mismo nivel de riesgo, a los que llegaron primero.

Esta “nueva” sala de emergencias que creamos, usa como base una cola de prioridad, por medio de un montículo binario mínimo (donde la clave más pequeña siempre está al frente). Para esto creamos la clase “MonticuloBinario” y “Paciente”, siendo montículo el montículo binario y paciente el “dato” que se ingresa en el montículo.

La clase “MonticuloBinario”, la definimos como una lista, la cual inicia con un único elemento, siendo este un infinito negativo (esto es necesario para el correcto funcionamiento de algunos métodos de la clase, pues para moverse por la lista se usa division o multiplicacion por 2, según el sentido del movimiento).

La clase “Paciente”, la definimos como un objeto que contiene nombre, apellido, el nivel de riesgo, la descripción del nivel de riesgo y el número de orden (por ej, el quinto en entrar recibe el número 5 y así).

La “nueva” sala de emergencias, incluye pausas entre que se atienden a los pacientes, para asemejarse a una sala de emergencias verdadera, y muestra por la terminal la lista de pacientes (aunque esta suele estar desordenada, se ordena al momento de atender, siendo el primero en la lista siempre el que tiene mayor prioridad).

Problema 2:

Aquí se nos pidió crear un registro de temperaturas dentro de un rango de fechas, que utilice internamente un árbol AVL.

Para esto creamos las clases “Arbol_AVL” y “Temperaturas_DB”, siendo la clase Arbol_AVL la forma de organizar estas temperaturas que serán ingresadas o manejadas (con manejadas nos referimos a trabajar con ellas según los métodos que nos pidieron crear) con la clase de Temperaturas_DB, la cual cuenta con los métodos pedidos por la cátedra.

La clase “Arbol_AVL”, para esta clase también fue necesario crear otra subclase del nodo de nuestro árbol llamada “NodoArbol”, la cual se inicializa con los datos de

quien es el padre, el hijo izquierdo y el hijo derecho (que en un comienzo todos son None), y su factor de equilibrio (iniciado en 0), así como su clave y valor. Esta clase se trata de un árbol binario de búsqueda, específicamente un árbol AVL, que mantiene al árbol equilibrado en todo momento haciendo uso del factor de equilibrio de cada nodo del árbol.

La clase "Temperaturas_DB", se inicializa como un árbol AVL, y le ingresamos temperaturas a dicho árbol con los siguientes datos: fecha y temperatura (en grados centígrados siempre), con el método de "guardar_temperatura". Esta clase también cuenta con métodos para ingresar una fecha y que nos devuelva la temperatura de dicha fecha, también podemos darle un rango de fechas y que nos diga la mayor temperatura registrada entre dichas fechas o la menor, así como ambas (los extremos digamos), podemos borrar temperaturas con solo ingresar la fecha de la temperatura a borrar, y por último podemos darle dos fechas y nos mostrará todas las temperaturas registradas entre esas fechas.

Métodos de Temperaturas_DB	Complejidad Big-O
guardar_temperatura	$O(\log n)$
devolver_temperatura	$O(\log n)$
max_temp_rango	$O(k + \log n)$
min_temp_rango	$O(k + \log n)$
temp_extremos_rango	$O(k + \log n)$
borrar_temperatura	$O(\log n)$
devolver_temperaturas	$O(k + \log n)$
cantidad_muestras	$O(1)$
temperaturas_en_rango	$O(k + \log n)$
nodos_en_rango	$O(k + \log n)$
recorrer_rango	$O(k + \log n)$

A continuación un análisis de estos órdenes de complejidad:

-guardar_temperatura: Inserta una nueva temperatura en el árbol AVL, y la inserción en AVL toma $O(\log n)$ debido al balanceo del árbol.

-devolver_temperatura: Busca una temperatura en el árbol AVL usando la fecha, y la búsqueda en AVL tiene un tiempo de $O(\log n)$.

-max_temp_rango: Recorre el árbol para encontrar nodos en un rango. El recorrido en AVL tiene un costo $O(k)$, donde k es el número de nodos en el rango, más $O(\log n)$ para encontrar los límites del rango.

-min_temp_rango: Igual que el método anterior, ya que realiza un recorrido en rango para buscar la mínima temperatura

-temp_extremos_rango: Hace un recorrido en rango para obtener el mínimo y máximo valor, por lo que tiene la misma complejidad que los métodos anteriores.

- borrar_temperatura**: Elimina una temperatura del árbol AVL, y la eliminación en AVL tiene una complejidad de $O(\log n)$.
- devolver_temperaturas**: Recorre el árbol en el rango dado para devolver las temperaturas, similar a los métodos anteriores de rango, con $O(k)$ por el número de nodos en el rango.
- cantidad_muestras**: Este método simplemente devuelve el tamaño del árbol, que se almacena como una variable. El acceso a esta variable toma $O(1)$.
- temperaturas_en_rango**: Este método llama a **nodos_en_rango**, y la complejidad está dominada por el recorrido en el rango que toma $O(k)$ más el tiempo para encontrar los límites $O(\log n)$.
- nodos_en_rango**: Realiza un recorrido en el árbol AVL para devolver los nodos en un rango, tomando $O(k)$ más el tiempo para encontrar los nodos iniciales, que es $O(\log n)$.
- recorrer_rango**: Este método realiza un recorrido en profundidad sobre los nodos que están en el rango, por lo que tiene un costo proporcional a k , que es el número de nodos en el rango.

Problema 3:

Para esta actividad teníamos que encontrar la forma de repartir un mensaje a todas las aldeas sin pasar nuevamente por las mismas, y cada aldea solo puede enviar mensajes a sus aldeas vecinas. La cátedra nos compartió el archivo con dichas aldeas y sus vecinos, nosotros usamos el algoritmo de grafos Prim para encontrar la ruta más eficiente para repartir la noticia.

Para lograr esto, rehusamos la clase "MonticuloBinario" y creamos las clases "Vértice" y "Grafo" pues son necesarias para el algoritmo Prim. Esto ya que Prim se basa en la construcción de un árbol de expansión de costo mínimo (AEM).

La clase "Vértice", modela cada nodo del grafo y almacena las conexiones con otros vértices junto con el peso de las aristas, cosa esencial para el algoritmo Prim ya que este selecciona la ruta con el menor peso para formar el árbol. A su vez también almacena la distancia mínima hasta el nodo en cuestión y el nodo desde el cual se llega a este con el menor peso (su predecesor).

La clase "Grafo", permite gestionar las aristas y vértices de todo el grafo. Así como agregar vértices de forma modular, asegurando la correcta definición de todos los nodos antes de calcular el árbol mínimo; también podemos agregar aristas, para conectar dos vértices con una arista de un peso especificado, esto es importante ya que el árbol de expansión mínima requiere la conexión de nodos sin dirección.

También incluye un método para cargar el grafo, que facilita la carga de datos para casos donde el grafo es grande y los datos deben ser cargados de manera eficiente para luego ser procesados por el algoritmo Prim.

El algoritmo Prim que creamos, se inicializa con el punto de partido en el vértice "inicio" (aldea Peligros), así como asigna a todos los vértices una distancia inicial de infinito y configura el predecesor de cada vértice como "None", asegurando que todos los vértices estén "desconectados". Este algoritmo usa la clase previamente

creada de “MonticuloBinario”, para mantener los vértices ordenados por su distancia mínima a medida que se va creando el árbol de expansión mínima, el montículo se inicializa con todos los vértices y permite extraer el vértice de menor costo en cada paso; optimizando el algoritmo al reducir el tiempo necesario para seleccionar el vértice con la menor distancia.